

FEASTIFY

A DYNAMIC DATABASE-DRIVEN DIGITAL RECIPE BOOK

Group Number: 41
Batch: 2024 (First Year, BICT)

W.M.N.D.Wicramasooriya (Index: 2803)
D.M.D.T.R.Dissanayaka (Index: 2718)



Table of Contents

1. PROJECT OVERVIEW.....	2
Purpose & Goals.....	2
Target Audience.....	2
Planned Core Features.....	2
2. SYSTEM DESIGN.....	3
Database Schema.....	3
Folder Structure.....	4
3. IMPLEMENTATION DETAILS.....	4
User Authentication	4
Contact Form Handler.....	4
Frontend-Backend Integration	4
4. CHALLENGES AND SOLUTIONS.....	6
5. INDIVIDUAL CONTRIBUTION	6
6. SETUP INSTRUCTIONS.....	6
7. REFERENCES	7

Final Project Report

Course: ICT 1209 – Web Technologies (2024 Batch, First Year, BICT)

Institution: Rajarata University of Sri Lanka

Group Number: 41

Project Title: Feastify: A Dynamic Database-Driven Digital Recipe Book

Group Members:

1. W.M.N.D.Wicramasooriya (Index No: 2803, Reg No: ITT/2024/118)

2. D.M.D.T.R.Dissanayaka (Index No: 2718, Reg No: ITT/2024/033)

Technology Stack: HTML5, CSS3, Bootstrap 5 Framework, Vanilla JavaScript, Native PHP 8, MySQL Relational Architecture.

1. PROJECT OVERVIEW

Purpose & Goals

The primary goal of Feastify is to transition from static web content to dynamic, database-driven rendering utilizing MySQL and PHP. The platform emphasizes role-based behavioral patterns: guest users are permitted to browse the public catalog, whereas authorized (authenticated) users possess the privileges to contribute, manage, and share recipes within the ecosystem.

Target Audience

The application is designed for daily home chefs, culinary enthusiasts, and students seeking an accessible, centralized repository for meal preparation and recipe discovery.

Planned Core Features

- **Dynamic Recipe Catalog & Query Filtering:** A searchable, category-based directory allowing users to filter recipes dynamically based on culinary categories and distinct keywords.
- **Secure Authentication & Dashboards:** A fully protected user environment requiring credential verification, granting access to personalized dashboards for recipe management.
- **Interactive Contact Form:** A robust communication interface bridging user inquiries to the database administrators.

2. SYSTEM DESIGN

Database Schema

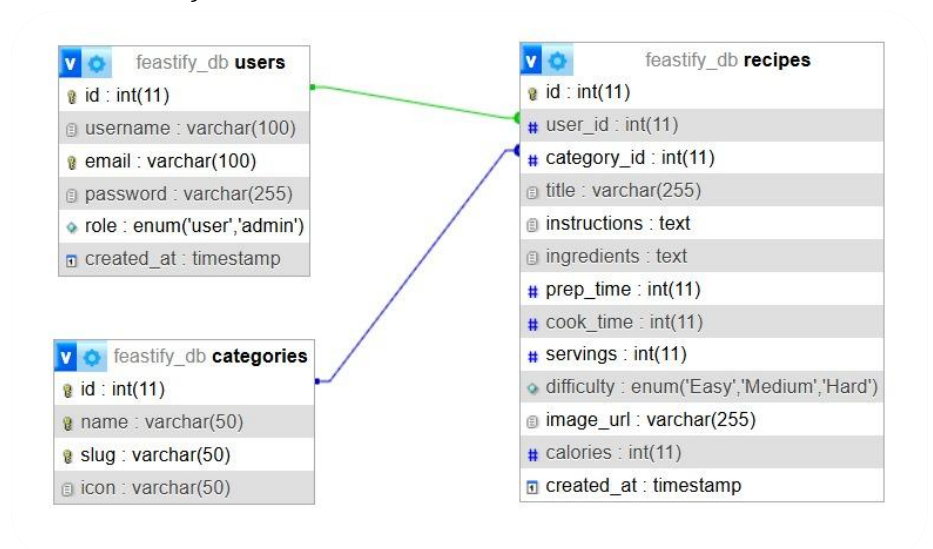
The relational database utilizes MySQL to manage application state. The architecture consists of the following structure:

- users
 - id (Primary Key, AUTO_INCREMENT)
 - username (VARCHAR)
 - email (VARCHAR, Unique)
 - password (VARCHAR, Hashed)
 - created_at (TIMESTAMP)
- messages
 - id (Primary Key, AUTO_INCREMENT)
 - name (VARCHAR)
 - email (VARCHAR)
 - message (TEXT)
 - created_at (TIMESTAMP)
- recipes
 - id (Primary Key, AUTO_INCREMENT)
 - user_id (Foreign Key linking to users.id)
 - category_id (Foreign Key linking to categories.id)
 - title (VARCHAR)
 - instructions (TEXT)
 - ingredients (TEXT)
 - prep_time (INT)
 - cook_time (INT)
 - servings (INT)
 - difficulty (ENUM)
 - image_url (VARCHAR)
 - calories (INT)
 - created_at (TIMESTAMP)
- categories
 - id (Primary Key, AUTO_INCREMENT)
 - name (VARCHAR)
 - slug (VARCHAR)
 - icon (VARCHAR)

id	: int(11)
username	: varchar(100)
email	: varchar(100)
password	: varchar(255)
role	: enum('user', 'admin')
created_at	: timestamp

id	: int(11)
name	: varchar(50)
slug	: varchar(50)
icon	: varchar(50)

id	: int(11)
user_id	: int(11)
category_id	: int(11)
title	: varchar(255)
instructions	: text
ingredients	: text
prep_time	: int(11)
cook_time	: int(11)
servings	: int(11)
difficulty	: enum('Easy', 'Medium', 'Hard')
image_url	: varchar(255)
calories	: int(11)
created_at	: timestamp



Folder Structure

The project directory strictly complies with university guidelines to ensure modularity and separation of concerns:

```
project/
├── css/
├── js/
├── images/
├── includes/
│   ├── db.php
│   └── functions.php
├── auth/
│   ├── register.php
│   ├── login.php
│   └── logout.php
├── contact.php
├── index.php
├── dashboard.php
├── README.md
└── database.sql
```

3. IMPLEMENTATION DETAILS

User Authentication

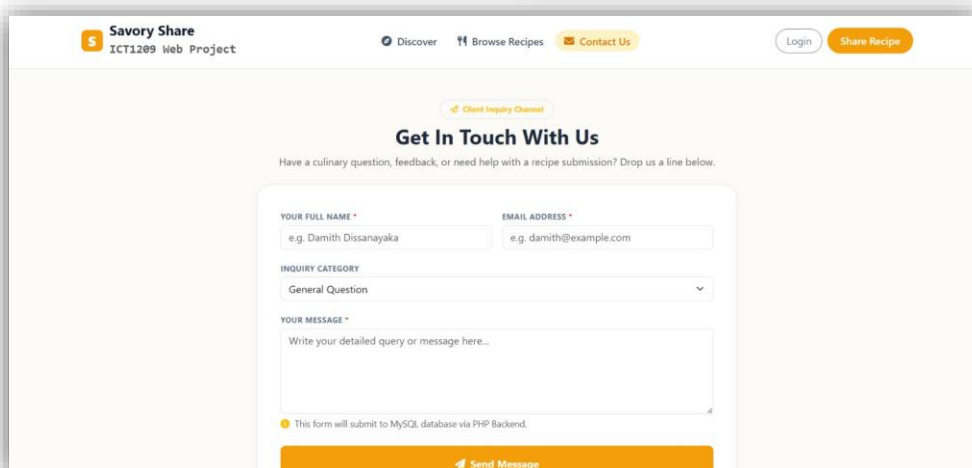
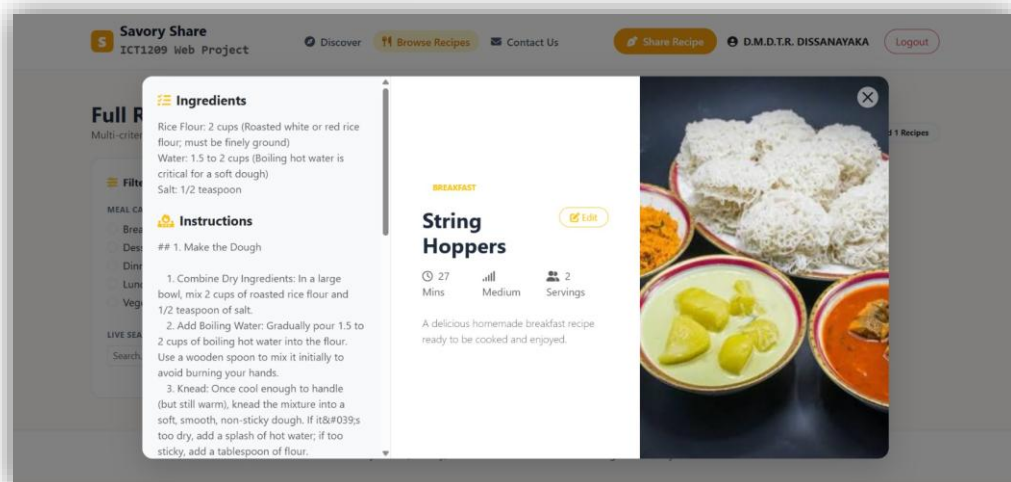
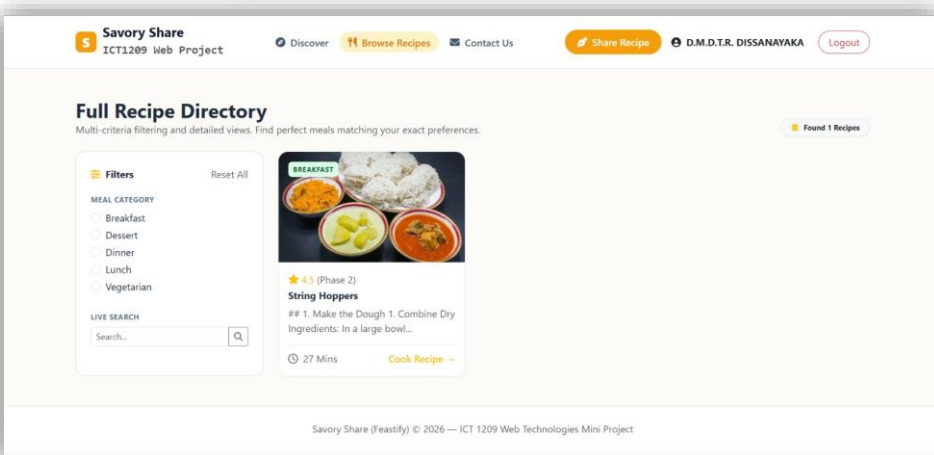
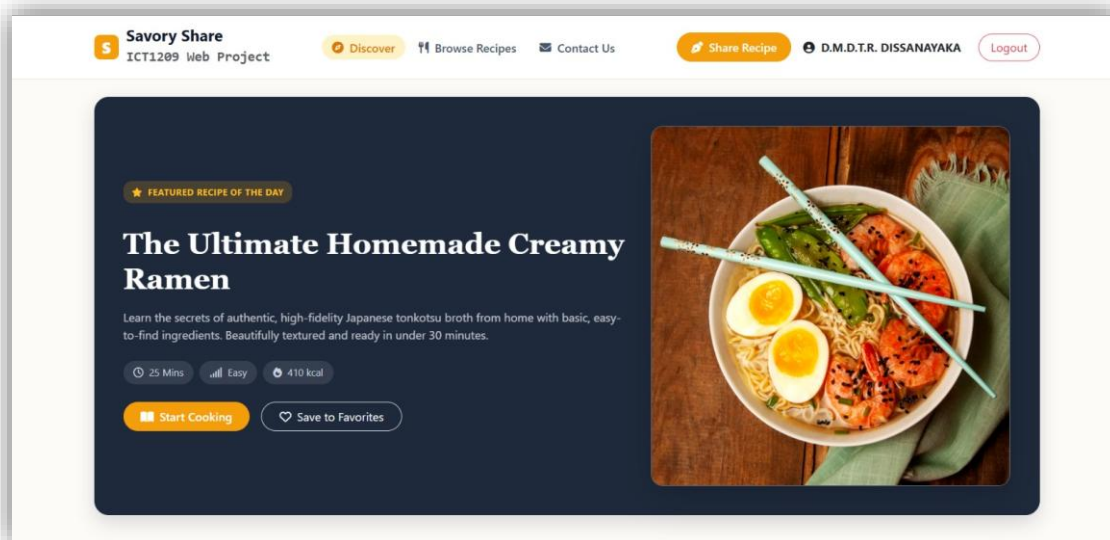
User credential security is enforced by hashing incoming passwords via the native `password_hash()` function utilizing the `PASSWORD_BCRYPT` algorithm. Secure login state tracking is maintained utilizing PHP Sessions. To mitigate session fixation vulnerabilities, `session_regenerate_id(true)` is strictly invoked upon successful authentication. Absolute session destruction is achieved in `logout.php` via `session_unset()` and `session_destroy()`.

Contact Form Handler

The contact module integrates server-side PHP validation logic to ensure data integrity prior to execution. User payloads are securely intercepted and inserted directly into the messages matrix using PDO Prepared Statements.

Frontend-Backend Integration

A dual-layer validation strategy is applied. The system preserves Vanilla JavaScript validation alerts on the client-side for immediate user feedback, acting alongside strict server-side validation. All database transactions are strictly channeled through PDO or MySQLi prepared statements to neutralize SQL Injection vulnerabilities entirely.



4. CHALLENGES AND SOLUTIONS

1. Challenge: Preventing SQL Injection.

- Solution: Entirely deprecated direct SQL string interpolation in favor of parameterized queries and PDO prepared statements for all database CRUD operations.

2. Challenge: Session Hijacking & Fixation.

- Solution: Enforced `session_regenerate_id(true)` immediately upon successful user login to cycle the session token securely.

3. Challenge: Dynamic UI Rendering Without Broken States.

- Solution: Combined Vanilla JS event handlers with server-side PHP conditional rendering to ensure data consistency without breaking the responsive interface during DOM updates.

5. INDIVIDUAL CONTRIBUTION

To maintain academic integrity, project responsibilities were distributed equitably between both group members:

- **W.M.N.D.Wicramasooriya:** Designed the structural MySQL database architecture, implemented secure session mechanisms in `login.php` and `logout.php`, configured the centralized `db.php` configuration layer, and enforced strict server-side validation properties and designed the conceptual Relational Database ER Diagram.
- **D.M.D.T.R.Dissanayaka:** Engineered the responsive UI components leveraging Bootstrap 5, implemented real-time Vanilla JavaScript form validation parameters, built the client-side user catalog search workflows, and coordinated frontend-backend view integration and managed the category-based filtering query logic integration.

6. SETUP INSTRUCTIONS

To evaluate and run the project locally, please follow these sequential steps:

1. Move the unzipped project directory into your XAMPP htdocs directory (`C:\xampp\htdocs\`).
2. Launch the XAMPP Control Panel and start both the Apache and MySQL modules.
3. Access phpMyAdmin via the browser (`http://localhost/phpmyadmin`), create a new database matching the theme requirements, and import the provided `database.sql` script to generate tables dynamically.
4. Launch the application in a local web browser by navigating to `http://localhost/[project_folder]`.

7. REFERENCES

- Official PHP Documentation Portal. (n.d.). Session Management & password_hash API parameters. Retrieved from <https://www.php.net>
- Bootstrap Core Team. (2023). Bootstrap v5.3 Component Layout Guidelines. Retrieved from <https://getbootstrap.com>
- Oracle Corporation. (2024). MySQL 8.0 Reference Manual: Prepared Statements & Foreign Key Constraints. Retrieved from <https://dev.mysql.com>